

SENIOR AI ENGINEER COMPLETE MASTER HANDBOOK

MCP · Tool Servers · LangGraph · AutoGen · CrewAI

Agentic AI · Advanced RAG · Multimodal · Fine-Tuning · GCP · Graph DBs

System Design · 300+ Interview Questions · Enterprise Architecture

Principal AI Architect · Staff Engineer · GenAI Platform Engineer

Production-Grade · Enterprise-Ready · Interview-Proven

TABLE OF CONTENTS

Chapter 1 — MCP — Model Context Protocol

- › Architecture & Lifecycle
- › Transport Layers
- › Tool Discovery & Registration
- › Authentication & Security
- › Enterprise Patterns
- › MCP vs Alternatives
- › 50 Interview Questions

Chapter 2 — Tool Server Integration

- › Function Calling Internals
- › Tool Execution Pipeline
- › Async & Long-Running Tools
- › Human-in-the-Loop
- › OpenTelemetry Observability

Chapter 3 — LangGraph

- › Graph-Based Orchestration
- › State Machines & DAGs
- › Checkpointing & Persistence
- › Multi-Agent Orchestration
- › Production Deployment
- › 50 Interview Questions

Chapter 4 — AutoGen

- › Multi-Agent Collaboration
- › Group Chat Architecture
- › Reflection & Self-Improvement
- › Enterprise Governance
- › AutoGen vs Alternatives

Chapter 5 — CrewAI

- › Roles, Tasks & Crews
- › Sequential & Hierarchical Workflows
- › Enterprise Automation
- › Failure Recovery

Chapter 6 — Agentic AI — Advanced

- › ReAct, Reflexion, Tree of Thoughts
- › Planning Algorithms
- › Memory Systems & World Models
- › Agent Governance & Security

Chapter 7 — Advanced RAG

- › RAG Taxonomy (Graph, Agentic, Hybrid, Self-RAG)
- › Chunking & Embedding Strategy

- › Reranking & Hybrid Search
- › Vector DB Comparison
- › Multi-Tenant RAG

Chapter 8 — Multimodal AI

- › Vision-Language Models
- › Document AI & OCR
- › Multimodal Pipelines
- › GPU Optimization

Chapter 9 — Fine-Tuning

- › SFT, RLHF, DPO, LoRA, QLoRA
- › Distributed Training
- › Catastrophic Forgetting & Alignment

Chapter 10 — GCP for AI Engineers

- › Vertex AI, GKE, Cloud Run
- › MLOps Pipelines
- › Cost & Security

Chapter 11 — Graph Databases & Knowledge Graphs

- › Neo4j & Cypher
- › GraphRAG
- › Hybrid Graph+Vector Search

Chapter 12 — System Design Case Studies

- › 10 Enterprise AI Architectures
- › Full Stack Tradeoff Analysis

Chapter 13 — Interview Preparation — 300+ Questions

- › Architecture · Debugging · Leadership · Coding Rounds

Chapter 14 — Must-Know Advanced Topics

- › vLLM, TensorRT-LLM, OpenTelemetry, AI Security, Guardrails, LLMOps

CHAPTER 1

MCP — Model Context Protocol

MCP (Model Context Protocol) is Anthropic's open standard that defines how AI models discover, invoke, and manage external tools and resources during inference. Announced in November 2024, MCP solves the N×M integration problem: without it, every LLM framework must write bespoke connectors to every tool. MCP provides a single, versioned wire protocol so any MCP-compliant client can speak to any MCP-compliant server.

1.1 Why MCP Was Introduced

Before MCP, tool integration was chaotic. OpenAI function calling, LangChain tools, Semantic Kernel plugins, and vendor APIs each had incompatible schemas. MCP draws lessons from Language Server Protocol (LSP) — the same idea that unified IDE-language tooling — and applies it to AI tool calling.

- Standardised JSON-RPC 2.0 wire format works across Python, TypeScript, Java, Go.
- Capability negotiation allows clients and servers to advertise supported features.
- Resource primitives expose files, DB rows, and API responses as typed context.
- Prompt primitives allow servers to ship reusable prompt templates.
- Sampling primitives let servers request LLM completions through the client.

1.2 MCP Architecture

MCP follows a client-host-server pattern. The HOST is the LLM application (e.g., Claude Desktop, your FastAPI service). The CLIENT lives inside the host and manages the protocol session. The SERVER exposes tools, resources, and prompts over a transport.



1.3 Transport Layers

MCP supports three transports, each suited to different deployment topologies:

Transport	Use Case	Latency	Auth	Notes
stdio	Local subprocess (CLI tools)	~1ms	OS process isolation	Default for desktop apps

Transport	Use Case	Latency	Auth	Notes
HTTP+SSE	Remote servers over internet	Network RTT	OAuth2 / API keys	Server-Sent Events for streaming
WebSocket	Bidirectional real-time	Network RTT	JWT / mTLS	Best for persistent sessions

1.4 MCP Lifecycle — Step by Step

- Step 1: Transport established — client spawns subprocess (stdio) or opens HTTP/WS connection.
- Step 2: initialize request — client sends protocol version, capabilities (roots, sampling).
- Step 3: initialize response — server replies with its capabilities (tools, resources, prompts).
- Step 4: initialized notification — client confirms handshake is complete.
- Step 5: tools/list — client discovers available tools and their JSON Schema input definitions.
- Step 6: tools/call — client sends tool name + arguments; server executes and returns result.
- Step 7: resources/read — client fetches typed context (files, DB rows) by URI.
- Step 8: Session close — either side sends CloseNotification or terminates transport.

1.5 Tool Discovery & Registration

Tool schemas use JSON Schema Draft 7. Every tool must declare name, description, and inputSchema. The description is critical — it is the primary signal the LLM uses for tool selection.

```
# Python MCP server – minimal tool registration
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

app = Server("enterprise-tools")

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="query_database",
            description="Execute a read-only SQL query against the analytics warehouse. "
            "Returns JSON rows. Use for: sales data, user metrics, cohort analysis.",
            input_schema={
                "type": "object",
                "properties": {
                    "sql": {
                        "type": "string",
                        "description": "SQL SELECT statement"
                    },
                    "limit": {
                        "type": "integer",
                        "default": 100,
                        "maximum": 1000
                    }
                },
                "required": ["sql"]
            }
        )
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "query_database":
        result = await run_query(arguments["sql"], arguments.get("limit", 100))
        return [TextContent(type="text", text=result)]
    raise ValueError(f"Unknown tool: {name}")

async def main():
    async with stdio_server() as (read, write):
        await app.run(read, write, app.create_initialization_options())
```

1.6 Authentication & Security

Remote MCP servers MUST enforce authentication. The recommended pattern uses OAuth 2.1 with PKCE for user-delegated access, or API keys with HMAC verification for service-to-service calls.

- Never trust tool arguments blindly — validate against JSON Schema before execution.
- Sandbox tool execution in isolated containers (gVisor, Firecracker) for untrusted code.
- Implement per-tool rate limiting to prevent LLM-driven resource exhaustion.
- Log every tool call with trace IDs for audit trails (SOC2, HIPAA requirements).
- Use short-lived credentials with automatic rotation via Vault or GCP Secret Manager.
- Implement tool allowlisting per user role — not every user should invoke DELETE operations.

```
# FastAPI MCP server with OAuth2 JWT auth from fastapi import FastAPI, Depends,
HTTPException from fastapi.security import OAuth2PasswordBearer import jwt, httpx
app_api = FastAPI() oauth2 = OAuth2PasswordBearer(tokenUrl="token") async def
verify_token(token: str = Depends(oauth2)): try: payload = jwt.decode(token, PUBLIC_KEY,
algorithms=["RS256"]) return payload except jwt.ExpiredSignatureError: raise
HTTPException(401, "Token expired") except jwt.InvalidTokenError: raise
HTTPException(401, "Invalid token") @app_api.post("/mcp/tools/call") async def
mcp_tool_call(body: dict, user=Depends(verify_token)): tool_name =
body["params"]["name"] if tool_name not in ALLOWED_TOOLS.get(user["role"], []): raise
HTTPException(403, f"Role {user['role']} cannot invoke {tool_name}") return await
dispatch_tool(tool_name, body["params"]["arguments"])
```

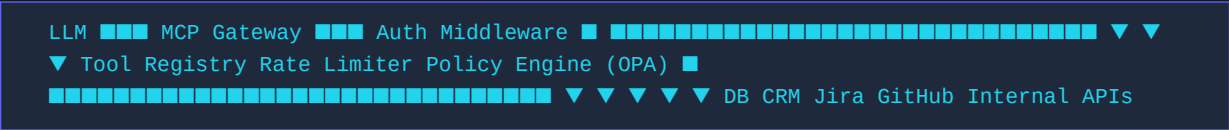
1.7 MCP vs Alternatives

Dimension	MCP	OpenAI Function Calling	LangChain Tools	OpenAPI / REST
Standard	Open (Anthropic)	Proprietary (OpenAI)	Framework-specific	Open (IETF)
Model agnostic	■ Yes	■ OpenAI only	■ LangChain only	■ Yes
Streaming	■ Native SSE	■ Yes	■ Varies	■ Yes
Resource primitives	■ Files, DB, APIs	■ No	■ No	■ No
Prompt primitives	■ Yes	■ No	■ PromptTemplate	■ No
Sampling (LLM calls)	■ Yes	■ No	■ No	■ No
Tool discovery	■ Dynamic	■ Static JSON	■ Static Python	■ OpenAPI spec
Enterprise adoption	Growing fast	Mature	Mature	Universal

1.8 Enterprise MCP Patterns

Pattern 1: MCP Gateway / Tool Registry

In large enterprises, you don't expose individual MCP servers directly to the LLM. Instead, deploy a central MCP Gateway that handles auth, rate limiting, routing, observability, and policy enforcement. Individual tool servers register via service discovery.



Pattern 2: Multi-Agent MCP Bus

Multiple AI agents share a common MCP bus. Agents can call tools AND call each other via an agent-as-tool pattern. The orchestrator agent discovers peer agents through the same tools/list mechanism.

1.9 Production Implementation — Docker + Kubernetes

```
# Dockerfile for MCP server FROM python:3.12-slim WORKDIR /app COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt COPY . . # Run over HTTP+SSE for K8s
deployment CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8080"] --- #
kubernetes/mcp-server.yaml apiVersion: apps/v1 kind: Deployment metadata: name:
mcp-database-server spec: replicas: 3 template: spec: containers: - name: mcp-server
image: gcr.io/myproject/mcp-db-server:v1.2 ports: - containerPort: 8080 env: - name:
DB_PASSWORD valueFrom: secretKeyRef: name: db-secret key: password resources: requests:
{cpu: "100m", memory: "128Mi"} limits: {cpu: "500m", memory: "512Mi"} livenessProbe:
httpGet: {path: /health, port: 8080} initialDelaySeconds: 5
```

1.10 MCP Interview Questions (Senior Level)

Q: Explain MCP's initialize handshake. What happens if capability versions mismatch?

A: The client sends an initialize request with its supported protocol version. The server responds with its version. If incompatible, the client MUST disconnect. Clients should implement graceful downgrade for minor version differences. This prevents silent incompatibility bugs in production.

Q: How would you design a multi-tenant MCP gateway serving 500 enterprise customers?

A: Use a shared gateway with per-tenant API key namespacing. Each tenant gets an isolated tool namespace. Rate limiting via Redis sliding window per (tenant, tool) pair. Audit logs to tenant-specific Cloud Storage buckets. OPA for per-tenant policy enforcement. Circuit breakers per upstream tool server.

Q: What are the security risks of giving an LLM access to MCP tools, and how do you mitigate them?

A: Risks: prompt injection via tool results, SSRF via URL tools, privilege escalation via chained tool calls, data exfiltration via subtle tool invocations. Mitigations: output sanitisation before returning tool results to LLM context, allowlist-based tool permissions per user role, read-only DB connections by default, tool call rate limits, human-in-the-loop approval for destructive operations.

Q: How does MCP handle long-running async tool execution?

A: MCP 0.5+ supports async tool execution via a two-phase pattern: tools/call returns an operation ID immediately, and the client polls tools/getResult or receives a notification via SSE/WebSocket. For very long operations, store state in Redis with TTL and expose progress via resources/read.

Q: Design an MCP server for a code execution sandbox. What isolation boundaries do you enforce?

A: Each execution spawns a gVisor-sandboxed container with: no network access, ephemeral filesystem (tmpfs), CPU/memory cgroup limits, 30s timeout enforced by SIGKILL, no privilege escalation, separate UID per invocation. Tool results are serialized through a strict schema — no arbitrary file system access returned.

CHAPTER 2

Tool Server Integration

Tool calling is the primary mechanism by which LLMs interact with the real world. Understanding the full execution pipeline — from LLM token generation to API response and back — is essential for building reliable production systems.

2.1 Function Calling Internals

When an LLM is given a tools array, it embeds the schemas into the context window during prefill. The model learns to generate structured JSON in a special tool-use token region rather than plain text. The inference server intercepts these tokens and routes to the tool executor.

```
# Full tool calling cycle with Anthropic SDK
import anthropic, json
client = anthropic.Anthropic()
tools = [{ "name": "get_stock_price", "description": "Get real-time stock price for a ticker symbol.", "input_schema": { "type": "object", "properties": { "ticker": { "type": "string", "description": "e.g. AAPL, GOOGL"}, "currency": { "type": "string", "enum": ["USD", "EUR"], "default": "USD" } }, "required": ["ticker"] } }]
messages = [{"role": "user", "content": "What is Apple's stock price?"}]
while True:
    resp = client.messages.create( model="claude-opus-4-5", max_tokens=1024, tools=tools, messages=messages )
    if resp.stop_reason == "end_turn":
        print(resp.content[0].text)
        break
    # Extract tool use blocks
    tool_uses = [b for b in resp.content if b.type == "tool_use"]
    if not tool_uses:
        break
    messages.append({"role": "assistant", "content": resp.content})
    tool_results = []
    for tu in tool_uses:
        # Execute the tool
        result = execute_tool(tu.name, tu.input)
        tool_results.append({ "type": "tool_result", "tool_use_id": tu.id, "content": json.dumps(result) })
    messages.append({"role": "user", "content": tool_results})
```

2.2 Tool Execution Pipeline Architecture

```
LLM Output (tool_use block) ── ▼ ──────────────────────────────────── Tool Router ── ← validates tool name exists ──────────────────────────────────── Schema ── ← validates arguments against JSON Schema ── Validator ── raises ToolInputError on failure ──────────────────────────────────── Auth/Policy ── ← checks user permissions, rate limits ── Middleware ── ──────────────────────────────────── Executor ── ← runs tool with timeout + circuit breaker ── (async pool) ── ──────────────────────────────────── Result ── ← serialises, truncates if >100KB ── Serialiser ── logs to observability stack ──────────────────────────────────── LLM Input (tool_result block)
```

2.3 Async & Long-Running Tools

Synchronous tool execution blocks the LLM turn. For operations >5 seconds (data exports, ML model runs, external API calls), use async execution with a polling or webhook pattern.

```
import asyncio from typing import Any import redis.asyncio as aioredis redis =
aioredis.from_url("redis://localhost") class AsyncToolExecutor: async def execute(self,
tool_name: str, args: dict) -> dict: job_id = generate_job_id() # Fire and forget
asyncio.create_task(self._run_job(job_id, tool_name, args)) return {"status": "pending",
"job_id": job_id, "poll_url": f"/tools/jobs/{job_id}"} async def _run_job(self, job_id:
str, tool: str, args: dict): try: result = await TOOL_REGISTRY[tool](**args) await
redis.setex(f"job:{job_id}", 3600, json.dumps({"status": "done", "result": result}))
except Exception as e: await redis.setex(f"job:{job_id}", 3600, json.dumps({"status":
"error", "message": str(e)})) async def poll_job(self, job_id: str) -> dict: raw = await
redis.get(f"job:{job_id}") return json.loads(raw) if raw else {"status": "not_found"}
```

2.4 Human-in-the-Loop Tools

For high-stakes operations (sending emails, executing trades, modifying production databases), inject a human approval step before the tool executes. LangGraph's interrupt mechanism or a custom approval queue both work well.

- Suspend graph execution at approval node; persist state to PostgreSQL.
- Send approval request to Slack/email with a signed approval URL (HMAC token).
- Resume graph execution after approval; proceed to tool call.
- Timeout unactioned approvals after N minutes; escalate or cancel.
- Log approval decisions with approver identity for compliance audit trail.

2.5 OpenTelemetry Integration for Tool Observability

```
from opentelemetry import trace from opentelemetry.sdk.trace import TracerProvider from
opentelemetry.sdk.trace.export import BatchSpanProcessor from
opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter provider =
TracerProvider() provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
trace.set_tracer_provider(provider) tracer = trace.get_tracer("tool-executor") async def
traced_tool_call(tool_name: str, args: dict) -> dict: with
tracer.start_as_current_span(f"tool.{tool_name}") as span:
span.set_attribute("tool.name", tool_name) span.set_attribute("tool.args_keys",
list(args.keys())) try: result = await TOOL_REGISTRY[tool_name](**args)
span.set_attribute("tool.success", True) span.set_attribute("tool.result_bytes",
len(str(result))) return result except Exception as e: span.record_exception(e)
span.set_attribute("tool.success", False) raise
```

LangGraph — Graph-Based Agent Orchestration

LangGraph is LangChain's stateful graph execution engine for building complex, multi-step AI workflows. Unlike simple chain-of-thought or sequential pipelines, LangGraph models your agentic workflow as a directed graph where nodes are processing units (LLM calls, tool executions, human reviews) and edges define control flow — including conditional branching and cycles.

3.1 Why LangGraph Exists

ReAct-style agents with simple loops break down at production scale. They lack: explicit state management, resumability after failures, human oversight checkpoints, and testable deterministic routing. LangGraph borrows from finite state machine theory and Pregel-style distributed graph computation.

3.2 Core Concepts

Concept	Definition	Analogy
StateGraph	Typed state container + node registry + edge definitions	Class definition
Node	Async function: State → partial State update	Microservice endpoint
Edge	Unconditional transition from node A to node B	Hard-wired pipe
Conditional Edge	Function that returns next node name based on state	Router/switch
Checkpoint	Persists state snapshots after each node	Database transaction log
Interrupt	Pause execution and wait for external input	Human approval gate
Thread	One execution instance of a graph; identified by task_id	Request session

3.3 Production LangGraph — Multi-Agent RAG Workflow

```
from langgraph.graph import StateGraph, END from langgraph.checkpoint.postgres import
PostgresSaver from typing import TypedDict, Annotated, Sequence import operator class
AgentState(TypedDict): messages: Annotated[Sequence[dict], operator.add] query: str
retrieved_docs: list[str] answer: str needs_human_review: bool iteration: int def
retrieve_node(state: AgentState) -> dict: docs =
vector_store.similarity_search(state["query"], k=5) return {"retrieved_docs":
[d.page_content for d in docs]} def generate_node(state: AgentState) -> dict: prompt =
build_rag_prompt(state["query"], state["retrieved_docs"]) response = llm.invoke(prompt)
return {"answer": response.content, "iteration": state["iteration"] + 1} def
quality_router(state: AgentState) -> str: if state["iteration"] > 3: return
"human_review" score = evaluate_answer(state["answer"], state["query"]) if score < 0.7:
return "retrieve" # Self-correcting loop return "end" def human_review_node(state:
AgentState) -> dict: # This node uses interrupt() – execution pauses here from
langgraph.types import interrupt decision = interrupt({"answer": state["answer"],
"query": state["query"]}) return {"answer": decision["approved_answer"]} # Build graph
builder = StateGraph(AgentState) builder.add_node("retrieve", retrieve_node)
builder.add_node("generate", generate_node) builder.add_node("human_review",
human_review_node) builder.set_entry_point("retrieve") builder.add_edge("retrieve",
"generate") builder.add_conditional_edges("generate", quality_router, {"retrieve":
"retrieve", "human_review": "human_review", "end": END})
builder.add_edge("human_review", END) # PostgreSQL checkpointing for durability
checkpointer = PostgresSaver.from_conn_string(DATABASE_URL) graph =
builder.compile(checkpointer=checkpointer)
```

3.4 Checkpointing & Resumability

Every node completion triggers a checkpoint write. State is serialized (JSON or pickle) and stored with (thread_id, checkpoint_id). On failure, re-run the graph with the same thread_id — LangGraph replays from the last successful checkpoint.

- PostgresSaver: production default — ACID, queryable, supports concurrent threads.
- RedisSaver: low-latency for short workflows, no persistence guarantee.
- SqliteSaver: development only, single-file, no concurrency.
- Custom savers: implement BaseCheckpointSaver — 3 methods: get, put, list.

3.5 LangGraph vs Alternatives

Dimension	LangGraph	CrewAI	AutoGen	Temporal	Airflow
Paradigm	State graph	Role-based crews	Conversational agents	Durable workflows	DAG scheduler
LLM-native	■ Yes	■ Yes	■ Yes	■ No	■ No
Cycles/loops	■ Yes	Limited	■ Yes	■ Yes	■ No
Human-in-loop	■ interrupt()	■ Manual	■ human_input_mode	■ signals	■ sensors
Persistence	■ Pluggable	■ Limited	■ Limited	■ Built-in	■ Built-in
Streaming	■ token-level	■ Limited	■ Limited	■ activity	■ No
Multi-agent	■ Subgraphs	■ Core design	■ GroupChat	■ orchestration	■ operators
Production maturity	High (2024)	Medium	Medium	Very High	Very High

3.6 Scaling LangGraph in Production

- Run graph execution as async tasks in a Celery/ARQ worker pool — decouple from HTTP request cycle.
- Use PostgreSQL connection pooling (PgBouncer) for high-throughput checkpointing (>1000 concurrent threads).
- Partition thread_ids by customer_id for database sharding; use consistent hashing.
- Cache static subgraphs (retrieval-only paths) with Redis TTL to reduce LLM calls.
- Deploy LangGraph API Server (LangServe) behind an API Gateway for managed auth + observability.
- Use Kafka as an event bus for inter-graph communication in multi-agent systems.

3.7 LangGraph Interview Questions

Q: When would you choose LangGraph over a simple ReAct loop?

A: LangGraph when you need: (1) cycles with state — e.g., self-correcting loops that reference previous iterations; (2) human-in-the-loop checkpoints that must survive process restarts; (3) parallel node execution; (4) complex conditional routing with >2 branches. ReAct is sufficient for single-agent, single-turn, stateless tasks.

Q: How do you handle a LangGraph node that intermittently fails due to a flaky external API?

A: Wrap the node in a retry decorator with exponential backoff (tenacity library). Configure the node to raise a RetryableError vs NonRetryableError — only retry on the former. Set max_retries=3. Because checkpointing saves state before each node, failed nodes restart from the last checkpoint without re-running successful upstream nodes.

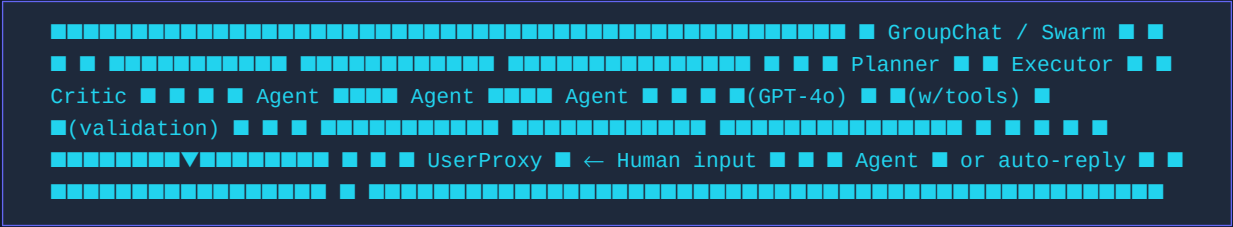
Q: Describe the architecture of a supervisor multi-agent system in LangGraph.

A: A Supervisor node receives the task and decides which specialist sub-agent to invoke next. Each sub-agent is a compiled subgraph, invoked as a node. The supervisor receives sub-agent output and decides: delegate to another agent, ask for human review, or terminate. State is shared via the parent graph's AgentState — sub-agents can only write to their designated state keys.

AutoGen — Conversational Multi-Agent Systems

Microsoft AutoGen (v0.4+ / AG2) is a framework for building multi-agent systems where agents collaborate through structured conversation. Unlike graph-based orchestration, AutoGen models agent interaction as a chat protocol — agents send and receive messages, and the framework manages the conversation topology.

4.1 AutoGen Core Architecture



4.2 Key Agent Types

Agent Type	Role	Tool Access	Human Input
AssistantAgent	LLM-powered reasoning	Configured tools	Never
UserProxyAgent	Executes code, human proxy	Code execution sandbox	Configurable
ConversableAgent	Base class for all agents	Custom	Custom
GroupChatManager	Selects next speaker	None (meta)	Never
SwarmAgent (v0.4)	Handoff-based routing	Per-agent	Never

4.3 Production AutoGen — Enterprise Coding Agent

```
import autogen from autogen import AssistantAgent, UserProxyAgent, GroupChat,
GroupChatManager config_list = [{"model": "claude-opus-4-5", "api_key": ANTHROPIC_KEY,
"base_url": "https://api.anthropic.com/v1"}] planner = AssistantAgent( name="Planner",
system_message="Decompose engineering tasks into subtasks. Output numbered plan. Never
write code.", llm_config={"config_list": config_list, "temperature": 0.1} ) engineer =
AssistantAgent( name="Engineer", system_message="Write production-quality Python code.
Follow PEP8, add type hints, handle exceptions.", llm_config={"config_list":
config_list, "temperature": 0.0} ) critic = AssistantAgent( name="Critic",
system_message="Review code for correctness, security, performance. Output APPROVED or
REVISION_NEEDED.", llm_config={"config_list": config_list, "temperature": 0.1} )
executor = UserProxyAgent( name="Executor", human_input_mode="NEVER",
code_execution_config={"work_dir": "/tmp/workspace", "use_docker": True, "docker_image":
"python:3.12-slim"}, max_consecutive_auto_reply=10 ) groupchat =
GroupChat(agents=[planner, engineer, critic, executor], messages=[], max_round=20)
manager = GroupChatManager(groupchat=groupchat, llm_config={"config_list": config_list})
executor.initiate_chat(manager, message="Build a JWT-validating FastAPI endpoint.")
```

4.4 AutoGen Security & Sandboxing

■ **Never run AutoGen's UserProxyAgent code execution without Docker sandboxing in production. LLM-generated code can be destructive.**

- Always set use_docker=True with a minimal base image (no network access).
- Bind-mount a temporary work directory; destroy it after each session.
- Set timeout limits on code execution (default 60s is often too long).
- Whitelist allowed Python packages in the Docker image — no pip install at runtime.
- Monitor resource usage (CPU, memory) with cgroups — prevent fork bombs.
- Log all executed code to immutable storage for audit.

4.5 AutoGen vs Alternatives

Dimension	AutoGen	LangGraph	CrewAI
Paradigm	Conversational agents	State graph	Role-based task crews
Strengths	Code generation, group debate	Complex stateful flows	Business process automation
Persistence	In-memory (v0.3), DB (v0.4)	Pluggable checkpoints	Limited
Code execution	■ Built-in Docker sandbox	■ Custom nodes needed	■ Custom
Human-in-loop	■ human_input_mode	■ interrupt()	■ callback
Learning curve	Medium	High	Low
Best for	Coding/research agents	Production workflows	Business automation

CrewAI — Role-Based Multi-Agent Teams

CrewAI models AI collaboration as a team of specialised agents with defined roles, each assigned tasks that collectively produce a deliverable. The mental model maps directly to human org charts — a researcher, a writer, a quality reviewer — making it intuitive for business stakeholders.

5.1 Core Primitives

Primitive	Description	Key Attributes
Agent	Role-specialised LLM entity	role, goal, backstory, tools, llm
Task	Unit of work assigned to an agent	description, expected_output, agent, context
Crew	Orchestrates agents + tasks	agents, tasks, process, verbose, memory
Tool	Capability an agent can invoke	name, description, func — any callable
Process	Sequential or Hierarchical	Sequential: tasks run in order; Hierarchical: manager delegates

5.2 Enterprise Proposal Generation System

```
from crewai import Agent, Task, Crew, Process from crewai_tools import SerperDevTool,
FileReadTool search_tool = SerperDevTool() file_tool = FileReadTool() # Define
specialist agents researcher = Agent( role="Senior Market Research Analyst",
goal="Gather comprehensive competitive intelligence and market data", backstory="15
years in B2B market research. Expert at synthesising " "public data into actionable
insights.", tools=[search_tool], llm="claude-opus-4-5", verbose=True, max_iter=5 #
Prevent infinite loops ) strategist = Agent( role="Solution Architect", goal="Design a
compelling technical solution tailored to client needs", backstory="Principal engineer
who has architected 50+ enterprise solutions.", tools=[file_tool], llm="claude-opus-4-5"
) writer = Agent( role="Proposal Writer", goal="Write a persuasive, client-specific
proposal document", backstory="Former McKinsey consultant specialising in tech RFPs.",
llm="claude-opus-4-5" ) # Define tasks with explicit context dependencies research_task
= Task( description="Research {company_name}'s tech stack, pain points, recent news,
competitors.", expected_output="Structured JSON: {company_profile, pain_points,
budget_signals, competitors}", agent=researcher ) solution_task = Task(
description="Design a solution addressing the identified pain points. Include
architecture diagram.", expected_output="Solution overview: components, timeline, team
structure, differentiators.", agent=strategist, context=[research_task] # Receives
researcher output ) proposal_task = Task( description="Write a 5-page executive proposal
incorporating research and solution design.", expected_output="Full proposal in Markdown
with exec summary, problem, solution, ROI, pricing.", agent=writer,
context=[research_task, solution_task], output_file="proposal_{company_name}.md" ) crew
= Crew( agents=[researcher, strategist, writer], tasks=[research_task, solution_task,
proposal_task], process=Process.sequential, verbose=2, memory=True # Enable cross-task
memory ) result = crew.kickoff(inputs={"company_name": "Acme Corp"})
```

5.3 Hierarchical Process Pattern

In the hierarchical process, a manager agent (automatically created by CrewAI) receives the overall goal, decides task assignments, delegates to worker agents, reviews outputs, and can re-delegate if quality is insufficient.

```
# Hierarchical crew – manager decides delegation order crew = Crew( agents=[researcher, strategist, writer], tasks=[research_task, solution_task, proposal_task], process=Process.hierarchical, manager_llm="claude-opus-4-5", # Manager agent uses this model verbose=True )
```

5.4 CrewAI Memory Architecture

- Short-term memory: in-context during a single task execution.
- Long-term memory: SQLite by default; swap to PostgreSQL for production.
- Entity memory: RAG-based memory over extracted entities (people, companies, facts).
- Contextual memory: combines all memory types into a unified retrieval query.

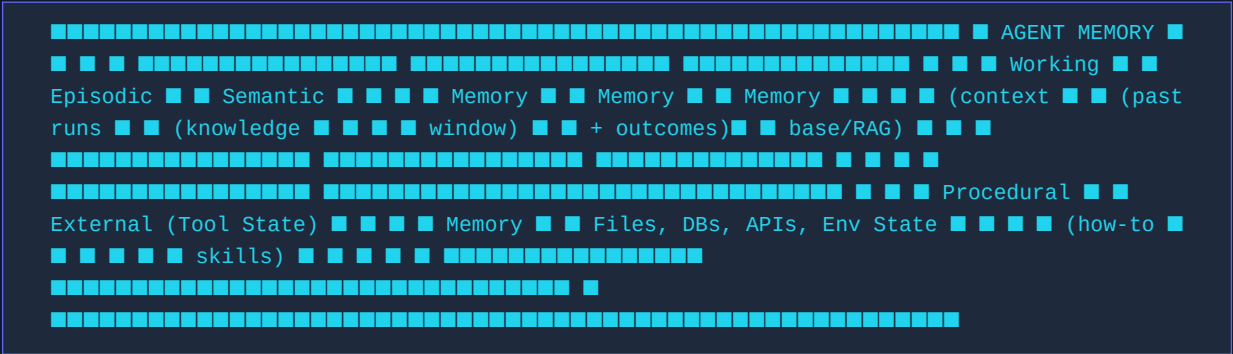
Agentic AI — Advanced Reasoning & Planning Systems

Agentic AI refers to LLM-powered systems that autonomously plan, execute multi-step actions, observe outcomes, and revise plans in pursuit of a goal — without explicit step-by-step human instruction. This is the frontier of AI engineering, requiring deep understanding of planning algorithms, memory systems, failure modes, and alignment.

6.1 Reasoning Paradigms

Paradigm	Mechanism	Strengths	Weaknesses
ReAct	Interleave Thought → Action → Observation	Simple, interpretable	No backtracking
Reflexion	ReAct + self-critique + memory of failures	Self-improving	Token expensive
Tree of Thoughts (ToT)	Branch multiple reasoning paths, CoS, DFS, Beam Search	Considers multiple paths	Very expensive
Graph of Thoughts (GoT)	Arbitrary DAG of thought nodes; Merge new problems	Handles complex problems	Complex to implement
LATS	ToT + MCTS rollouts + value function	Optimal search	Requires reward model
Chain of Thought (CoT)	Explicit reasoning steps in prompt	Wide applicability	Single path

6.2 Agent Memory Systems



6.3 Production Agent Failure Modes & Mitigations

Failure Mode	Root Cause	Mitigation
Infinite loop	Agent repeats same action	Max iterations + loop detection via state hash
Tool hallucination	LLM invents non-existent tool args	Strict JSON Schema validation + rejection
Context overflow	Long conversations exceed context window	Sliding window + summarisation compression

Failure Mode	Root Cause	Mitigation
Goal drift	Sub-goals replace original goal	Goal pinning in system prompt + periodic re-anchoring
Cascading failures	One tool failure breaks downstream actions	Circuit breakers + fallback strategies
Cost explosion	Unbounded LLM calls in loop	Token budget limits + cost guardrails per thread
Prompt injection	Malicious data from tools hijacks agent	Tool output sanitisation + separate trust levels

6.4 Agent Governance Framework

- Define agent capability tiers: read-only, write-limited, autonomous — assign based on risk.
- Implement an agent activity budget: max tokens, max tool calls, max wall-clock time per task.
- Deploy agent sandbox environments for testing before production promotion.
- Maintain an agent audit log: every decision, tool call, and state transition, immutably stored.
- Implement kill switches: admin API to terminate any agent thread immediately.
- Red-team agents regularly: attempt to make them perform unintended actions via adversarial inputs.

Advanced RAG — Retrieval-Augmented Generation at Scale

RAG has evolved far beyond the naive retrieve-then-generate pattern. Production RAG systems are sophisticated retrieval engines that combine dense and sparse search, multi-stage reranking, query transformation, self-correction, and graph traversal. Mastering RAG architecture is non-negotiable for senior AI engineers.

7.1 RAG Taxonomy

RAG Variant	Mechanism	Best For	Complexity
Naive RAG	Embed query → top-K vector search	Prototyping	Low
Advanced RAG	Query rewrite + hybrid search + reranking	Production	Medium
Modular RAG	Pluggable retrieval, reranking, fusion	Flexible systems	Medium
Graph RAG	Entity extraction → knowledge graph traversal	Relationship queries	High
Agentic RAG	Agent decides retrieval strategy, iterates	Complex, uncertain	High
Self-RAG	Model generates retrieval tokens	High quality answers	High
Corrective RAG (CRAG)	Evaluate retrieved docs, fallback	Accuracy critical	Medium
Fusion RAG	Multiple queries → multiple retrievals	Comprehensive coverage	Medium
Multi-hop RAG	Iterative retrieval: each hop uses previous	Multifaceted queries	High

7.2 Production RAG Pipeline Architecture

```

User Query ──► Query Analysis ──► Classify: factual / analytical / comparative
              │
              └─► Rewriting ──► Expand with HyDE or Step-Back prompting
                          │
                          └─► (multiple query variants)
                              │
                              └─► Hybrid Search ──► Metadata Filter ──► Dense (vector)
                                                                │
                                                                └─► date, source, doc_type, tenant
                                                                    │
                                                                    └─► Sparse (BM25)
                                                                        │
                                                                        └─► Reciprocal Rank ──► Fuse dense + sparse rankings
                                                                              │
                                                                              └─► Fusion (RRF)
                                                                                  │
                                                                                  └─► Cross-Encoder ──► Rerank top-50
                                                                                          │
                                                                                          └─► Reranker ──► (Cohere, BGE, Jina)
                                                                                              │
                                                                                              └─► Context ──► Compress to fit context window
                                                                                                      │
                                                                                                      └─► Summarise long docs
                                                                                                          │
                                                                                                          └─► Generation + LLM generates with citations
                                                                                                              │
                                                                                                              └─► Citation
                                                                                                                  │
                                                                                                                  └─► Faithfulness ──► Check: does answer follow from docs?
                                                                                                                      │
                                                                                                                      └─► Evaluation ──► Flag low-confidence answers
  
```

7.3 Chunking Strategies

Strategy	Description	Best For	Chunk Size
Fixed-size	Split every N tokens with overlap	Homogeneous docs	256–512 tokens
Sentence	Split at sentence boundaries	Narrative text	Variable
Recursive char	Try <code>\n\n</code> → <code>\n</code> → <code>.</code> → space	Code + prose mixed	512–1024 tokens
Semantic	Split when embedding similarity drops	Dense technical docs	Variable
Hierarchical	Parent chunks index child chunks	Long docs with sections	Parent:2048, Child:256
Late chunking	Embed full doc, then chunk embeddings	Preserves context	Variable
Agentic chunking	LLM identifies semantic boundaries	Complex structure	Variable

7.4 Vector Database Comparison

DB	Architecture	ANN Algorithm	Scalability	Filtering	Managed
Pinecone	Cloud-native serverless	Custom HNSW	Serverless ∞	Metadata filter	■ Yes
Weaviate	Multi-modal, graph-capable	HNSW	Horizontal sharding	GraphQL + BM25	■ Cloud
Qdrant	Rust, high perf	HNSW + IVF	Distributed cluster	JSON payload filter	■ Cloud
Milvus	Distributed, Kubernetes native	HNSW, DiskANN	Billion-scale	Scalar + vector	■ Self-hosted
Chroma	Embedded or server	HNSW (hnsplib)	Single-node	Metadata filter	■ Self-hosted
FAISS	Library (not a DB)	IVF, HNSW, PQ	Single-node (lib)	None native	■ Library
pgvector	PostgreSQL extension	HNSW + IVF	PostgreSQL limits	SQL WHERE	■ Cloud SQL

7.5 Hybrid Search Implementation

```
from qdrant_client import QdrantClient, models from rank_bm25 import BM25Okapi import numpy as np client = QdrantClient("localhost", port=6333) embedder = load_embedding_model("text-embedding-3-large") def hybrid_search(query: str, top_k: int = 20, rrf_k: int = 60) -> list[dict]: # 1. Dense search (vector similarity) query_vec = embedder.encode(query) dense_results = client.search( collection_name="docs", query_vector=query_vec, limit=top_k, with_payload=True ) # 2. Sparse search (BM25 on stored tokenised corpus) bm25_scores = bm25_index.get_scores(query.split()) sparse_top = np.argsort(bm25_scores)[-top_k:][::-1] # 3. Reciprocal Rank Fusion fused = {} for rank, hit in enumerate(dense_results): doc_id = hit.payload["doc_id"] fused[doc_id] = fused.get(doc_id, 0) + 1/(rrf_k + rank + 1) for rank, idx in enumerate(sparse_top): doc_id = corpus_ids[idx] fused[doc_id] = fused.get(doc_id, 0) + 1/(rrf_k + rank + 1) # Sort by fused score, return top_k sorted_docs = sorted(fused.items(), key=lambda x: x[1], reverse=True)[:top_k] return [fetch_doc(doc_id) for doc_id, _ in sorted_docs] def rerank(query: str, docs: list[dict], top_n: int = 5) -> list[dict]: import cohere co = cohere.Client(COHERE_API_KEY) results = co.rerank( model="rerank-english-v3.0", query=query, documents=[d["content"] for d in docs], top_n=top_n ) return [docs[r.index] for r in results.results]
```

7.6 Graph RAG Architecture

Graph RAG (Microsoft Research, 2024) combines knowledge graph construction with vector retrieval. It excels at questions requiring relationship traversal — 'Who reported to whom?', 'Which products share components?' — that pure vector search cannot answer.

```
# Graph RAG pipeline (simplified) # Phase 1: Indexing – build knowledge graph from
langchain_experimental.graph_transformers import LLMGraphTransformer from
langchain_community.graphs import Neo4jGraph graph = Neo4jGraph(url=NEO4J_URI,
username="neo4j", password=NEO4J_PASS) transformer = LLMGraphTransformer(llm=claude) #
Extract entities and relationships from documents graph_docs =
transformer.convert_to_graph_documents(documents) graph.add_graph_documents(graph_docs,
include_source=True) # Phase 2: Community detection (Leiden algorithm) # Groups tightly
connected nodes into communities # Each community gets a summary → stored as vector
embedding # Phase 3: Query – local + global search def graph_rag_query(question: str,
mode: str = "local") -> str: if mode == "global": # Search community summaries – answers
broad thematic questions return query_community_summaries(question) else: # Subgraph
traversal from matched entities – precise relationship queries entities =
extract_entities(question) subgraph = graph.query(f""" MATCH (n)-[r*1..3]-(m) WHERE
n.name IN {entities} RETURN n, r, m LIMIT 100 """) return generate_answer(question,
subgraph)
```

Multimodal AI — Vision, Audio & Document Intelligence

Multimodal AI systems process and reason across multiple data modalities simultaneously. For enterprise AI engineers, the most critical capabilities are document intelligence (PDF, forms, tables), vision-language reasoning (chart analysis, UI understanding), and multimodal RAG pipelines.

8.1 Vision-Language Model Landscape

Model	Provider	Context	Strengths	Best Use Case
Claude 3.5 Sonnet	Anthropic	200K tokens	Document analysis, charts	Enterprise document AI
GPT-4o	OpenAI	128K tokens	General vision, real-time audio	Multimodal chat
Gemini 1.5 Pro	Google	2M tokens (incl. video)	Long video, Google Workspace	Video understanding
LLaVA / InternVL	Open source	4–32K tokens	Self-hosted, fine-tunable	Private data
Qwen-VL	Alibaba	4K tokens	OCR, Chinese documents	Asian document AI

8.2 Enterprise Document AI Pipeline

```
import anthropic, base64 from pathlib import Path client = anthropic.Anthropic() def
extract_document_data(pdf_path: str) -> dict: """Extract structured data from a complex
PDF with tables, charts, text.""" # Convert PDF pages to images images =
convert_pdf_to_images(pdf_path, dpi=200) all_extractions = [] for i, img in
enumerate(images): img_b64 = base64.standard_b64encode(img).decode() response =
client.messages.create( model="claude-sonnet-4-6", max_tokens=4096, messages=[{ "role":
"user", "content": [ { "type": "image", "source": {"type": "base64", "media_type":
"image/png", "data": img_b64} },{ "type": "text", "text": """Extract ALL information
from this document page. Return as JSON: { "text_blocks": [...], "tables": [{"headers":
[], "rows": [[]]}, "key_values": {"field": "value"}, "figures": [{"caption": "...",
"description": "...}"], "page_type": "form|table|narrative|mixed" } Be exhaustive.
Include every number, date, name, and value.""" } ] ] ) import json extraction =
json.loads(response.content[0].text) extraction["page"] = i + 1
all_extractions.append(extraction) return {"pages": all_extractions, "total_pages":
len(images)}
```

8.3 GPU Optimization for Multimodal Inference

- Use Flash Attention 2 for vision transformers — 2–4x memory reduction on long sequences.
- Quantize image encoders to INT8 with ONNX Runtime — 3x throughput, <1% accuracy loss.
- Batch image preprocessing on GPU with torchvision transforms — remove CPU bottleneck.

- For high-throughput document processing: use vLLM's vision model support with PagedAttention.
- Cache image embeddings for repeated processing (same PDF, multiple queries) — 10x speedup.
- Use NVIDIA TensorRT to compile vision encoder for target GPU (A100/H100) — 2x inference speedup.

Fine-Tuning — From LoRA to RLHF at Production Scale

Fine-tuning adapts a pre-trained model's weights for a specific domain, task format, or style. Understanding when to fine-tune vs. when to use prompt engineering or RAG is a critical architectural decision that separates senior engineers from junior ones.

9.1 Fine-Tuning Decision Framework

Approach	When to Use	Cost	Latency	Data Needed
Prompt Engineering	General tasks, rapid iteration	Zero	Base model	None
Few-shot Prompting	Consistent format, small examples fit context	Zero	Base model	5–50 examples
RAG	Knowledge grounding, frequently updated info	+retrieval	+retrieval	Documents
SFT (LoRA)	Specific format/style/domain, strict adherence	Medium	Same	100–10K examples
Full Fine-tune	Complete domain shift, custom tokenisation	High	Same	100K+ examples
RLHF/DPO	Alignment, preference learning, safety	Very High	Same	Human preferences

9.2 LoRA & QLoRA — Theory and Implementation

LoRA (Low-Rank Adaptation) freezes pre-trained weights W and injects trainable low-rank matrices A and B such that the adapted weight is $W + BA$ where r is much less than $\min(d,k)$. QLoRA combines 4-bit quantization of the base model with LoRA adapters in 16-bit — enabling fine-tuning of 70B models on a single A100.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, TrainingArguments
from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer
import torch

# Load base model in 4-bit (QLoRA)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8B-Instruct",
    load_in_4bit=True,
    torch_dtype=torch.float16,
    device_map="auto",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    # Nested quantization
    bnb_4bit_quant_type="nf4" # NormalFloat4
) # LoRA config – target attention projection layers
lora_config = LoraConfig(
    r=16, # Rank – higher = more capacity
    lora_alpha=32, # Scaling factor (alpha/r = effective learning rate scale)
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # Should be ~1-2% of total params

training_args = TrainingArguments(
    output_dir="./checkpoints",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4, # Effective batch = 16
    warmup_steps=100,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=10,
    evaluation_strategy="steps",
    eval_steps=200,
    save_strategy="steps",
    save_steps=200,
    load_best_model_at_end=True,
    optim="paged_adamw_32bit" # Memory-efficient optimizer
)

trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    dataset_text_field="text",
    max_seq_length=2048,
    args=training_args
)
trainer.train()
model.save_pretrained("./fine_tuned_adapter")
```

9.3 DPO — Direct Preference Optimisation

DPO eliminates the need for a separate reward model (unlike RLHF). It directly optimises the language model using preference pairs (chosen, rejected) via a closed-form objective derived from the Bradley-Terry preference model. DPO is 2–3x cheaper than RLHF and achieves comparable alignment results on most tasks.

```
from trl import DPOTrainer, DPOConfig

# Dataset format: {"prompt": "...", "chosen": "...", "rejected": "..."}
dpo_config = DPOConfig(
    beta=0.1, # Temperature – higher = more conservative
    loss_type="sigmoid", # Standard DPO loss
    learning_rate=5e-7,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    num_train_epochs=1,
    max_length=1024,
    max_prompt_length=512
)
dpo_trainer = DPOTrainer(
    model=sft_model, # Start from SFT checkpoint
    ref_model=ref_model, # Frozen reference (original SFT)
    args=dpo_config,
    train_dataset=preference_dataset,
    tokenizer=tokenizer
)
dpo_trainer.train()
```

9.4 Distributed Training — DeepSpeed ZeRO

ZeRO Stage	What's Partitioned	Memory Reduction	Communication Overhead
Stage 1	Optimizer states	4x	Low
Stage 2	Optimizer states + gradients	8x	Medium
Stage 3	Optimizer states + gradients + parameters	16x	High
ZeRO-Infinity	Stage 3 + NVMe offload	Unlimited	Very High

GCP for AI Engineers — Vertex AI to Production

10.1 GCP AI Service Map

10.2 Deploy Scalable RAG on GCP — Architecture

10.3 Vertex AI Pipeline for MLOps

CHAPTER 11

Graph Databases & Knowledge Graphs

Graph databases model data as nodes and relationships, enabling traversal queries that are expensive or impossible in relational databases. For AI engineers, the primary use case is knowledge graph construction for Graph RAG, entity resolution, and relationship-based reasoning.

11.1 Neo4j & Cypher Essentials for AI Engineers

```
// Create entity nodes and relationships (extracted by LLM) CREATE (apple:Company {name:
"Apple Inc", founded: 1976, sector: "Technology"}) CREATE (tim:Person {name: "Tim Cook",
title: "CEO"}) CREATE (iphone:Product {name: "iPhone", category: "Smartphone"}) CREATE
(tim)-[:LEADS {since: 2011}]->(apple) CREATE (apple)-[:MANUFACTURES]->(iphone) //
Multi-hop graph traversal for Graph RAG // "Which executives lead companies that make AI
chips?" MATCH (p:Person)-[:LEADS]->(c:Company)-[:MANUFACTURES]->(prod:Product) WHERE
prod.category = "AI Chip" RETURN p.name, c.name, prod.name ORDER BY c.revenue DESC //
Vector similarity search on node embeddings (Neo4j 5.11+) MATCH (d:Document) WHERE
d.embedding IS NOT NULL CALL db.index.vector.queryNodes('document_embeddings', 10,
$query_embedding) YIELD node, score RETURN node.content, score ORDER BY score DESC
```

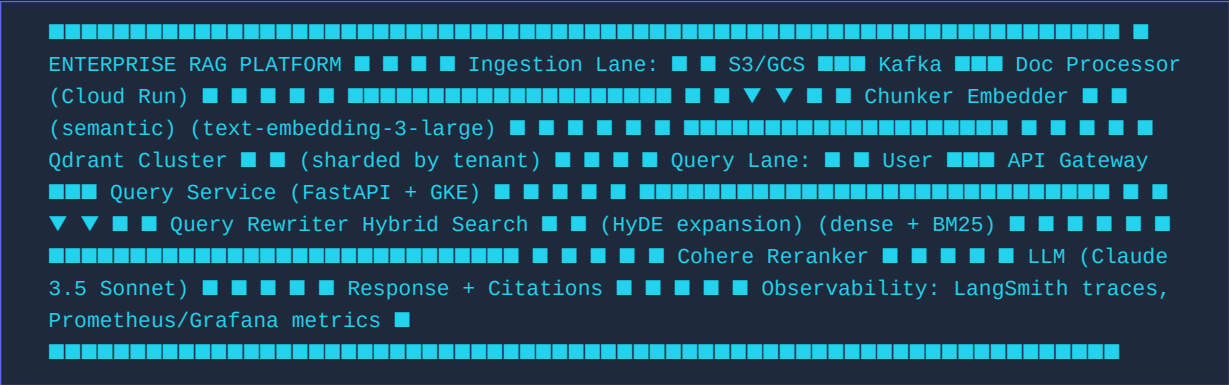
11.2 Enterprise Knowledge Graph + RAG Architecture

```
from langchain_community.graphs import Neo4jGraph from langchain_anthropic import
ChatAnthropic graph = Neo4jGraph(url=NEO4J_URI, username="neo4j", password=NEO4J_PASS)
llm = ChatAnthropic(model="claude-sonnet-4-6") def graph_rag_query(question: str) ->
str: # Step 1: Extract entities from question using LLM entity_prompt = f"""Extract named
entities (companies, people, products) from: "{question}" Return as JSON list:
["entity1", "entity2"]""" entities = json.loads(llm.invoke(entity_prompt).content) #
Step 2: Traverse knowledge graph from matched entities cypher = f""" MATCH (n) WHERE
n.name IN {entities} CALL apoc.path.subgraphAll(n, {{maxLevel: 2, relationshipFilter:
"LEADS|OWNS|ACQUIRES"}}) YIELD nodes, relationships RETURN nodes, relationships LIMIT 50
""" subgraph = graph.query(cypher) # Step 3: Serialize graph context + generate answer
context = format_graph_context(subgraph) return
llm.invoke(f"Context:\n{context}\n\nQuestion: {question}").content
```

System Design Case Studies for Senior AI Engineers

System design interviews at senior AI engineer level require you to architect complete, production-ready systems including data flow, scaling strategy, failure handling, cost model, and monitoring. Here are 10 canonical enterprise AI system designs.

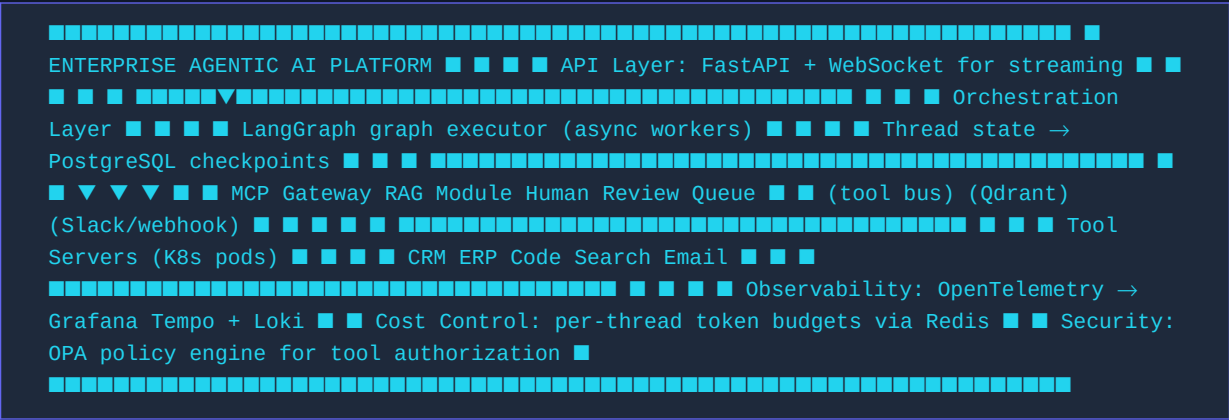
12.1 Enterprise RAG Platform



Key Design Decisions & Tradeoffs

Decision	Choice	Rationale	Alternative
Vector DB	Qdrant cluster	Best perf/cost at 10M+ docs, Rust-based	Redis-based (simpler ops)
Chunking	Hierarchical parent/child	Precise retrieval + broad context	Semantic (slower)
Reranker	Cohere rerank-v3	Best quality, low latency	BGE-reranker (self-hosted)
Embedding	text-embedding-3-large	Best quality, 3072-dim	Gemini (if GCP-native)
Auth	JWT + tenant namespace	Multi-tenant isolation	API key per tenant
Caching	Redis semantic cache	Reduce LLM costs 40-60%	No cache (freshness)

12.2 Agentic AI Platform Architecture



12.3 AI Customer Support System

Requirements

- Handle 10K concurrent sessions
- P99 response latency < 2s
- Escalation to human agents with full context
- Multi-language support (50+ languages)
- Full audit trail for compliance
- Cost target: <\$0.02 per conversation turn

Architecture Decisions

Layer	Technology	Scaling Strategy
Session management	Redis Cluster	Consistent hash ring, 10K sessions/node
LLM routing	LiteLLM proxy	Smart routing: Claude for complex, Haiku for simple
Intent classification	Fine-tuned BERT (self-hosted)	CPU inference, 5ms latency
Vector search	Pinecone serverless	Serverless auto-scaling
Escalation queue	PostgreSQL + Pub/Sub	Priority queue, FIFO per customer tier
Analytics	BigQuery streaming	Real-time CSAT, resolution rate dashboards

Interview Preparation — 300+ Senior-Level Questions

This chapter provides the most comprehensive collection of senior AI engineer interview questions, organised by category, with model answers, follow-up questions, and common mistakes. Use these for both preparation and as a reference for conducting technical interviews.

13.1 RAG Architecture Questions

Q: Design a RAG system for 10 million documents with sub-200ms P99 latency. Walk me through your architecture.

A: Use Qdrant distributed cluster (3 nodes) with HNSW index. Pre-compute and cache embeddings. Use approximate nearest neighbor (not exact) search with ef=64. Implement a 2-tier cache: L1 Redis for recent queries, L2 semantic cache using cosine similarity. Use batched embedding with GPU. Deploy the query service on GKE with HPA on CPU/RPS. Expected: embedding ~20ms, ANN search ~15ms, reranking ~50ms, LLM ~80ms = ~165ms P99.

Q: How do you handle documents that are updated or deleted in a production RAG system?

A: Implement incremental indexing with a change data capture (CDC) stream from the document store. Use Kafka to propagate updates to the vector index. For updates: delete old vector by doc_id, re-embed and insert new version. For deletes: soft-delete with a 'deleted' metadata flag, filter in retrieval. Run a reconciliation job nightly to hard-delete tombstoned vectors. Use document versioning to handle concurrent updates.

Q: What is the difference between RAG faithfulness and relevance, and how do you measure them in production?

A: Faithfulness: does the generated answer follow from the retrieved context? Measure with an LLM judge that checks each claim in the answer against retrieved docs (RAGAS framework). Relevance: are the retrieved documents relevant to the question? Measure with another LLM judge or a trained retrieval quality model. In production: run both on a sample (5%) of traffic; alert when faithfulness drops below 0.85 or relevance below 0.80.

Q: When would you use Graph RAG instead of standard vector RAG?

A: Graph RAG excels at: (1) relationship queries — 'who reports to whom', 'which products share suppliers'; (2) multi-hop reasoning — 'companies that own companies that supply to Tesla'; (3) thematic summarisation across large corpora. Standard vector RAG is better for: semantic similarity — 'find documents similar to this one'; factual lookup — 'what did the Q3 report say about revenue'. Graph RAG costs 3-5x more to build and maintain; only worth it if your query patterns are relationship-heavy.

13.2 Agentic AI & Multi-Agent Questions

Q: How do you prevent an autonomous agent from taking destructive actions in production?

A: Defence in depth: (1) Tool-level: write operations require confirmation tool call before execution; (2) Policy-level: OPA rules enforce what actions are permitted per agent role; (3) Infra-level: DB connections are read-only by default; write connections require explicit escalation; (4) Monitoring-level: anomaly detection on tool call patterns — alert on unusual sequences; (5) Human-in-the-loop: any action with side-effects > threshold triggers async human approval via Slack. Never trust agent-declared permissions — enforce at infrastructure layer.

Q: Design a multi-agent system where a researcher agent and a writer agent collaborate to produce a research report.

A: Use LangGraph supervisor pattern. Supervisor receives topic, creates a plan. Researcher subgraph: iterative retrieval (Tavily web search + internal RAG) with self-critique loop — continues until confidence score >0.8. Writer subgraph: receives research output, generates report sections in parallel (LangGraph parallel nodes), then merges. Quality gate: critic agent scores the draft; if score <0.7, sends specific feedback back to writer for revision (max 2 revisions). Final output to human review node before delivery.

Q: How do you measure the quality of an autonomous agent's performance?

A: Define task-specific metrics: (1) task completion rate — % of tasks fully resolved without human escalation; (2) step efficiency — actual steps / minimum steps required (lower is better); (3) tool call accuracy — % of tool calls that returned useful results vs. errors; (4) cost per task — total tokens * price; (5) time to completion. For end-to-end quality: use LLM-as-judge with a rubric, or human evaluation on a sample. Run benchmarks against a fixed test suite of tasks after each model/prompt change.

13.3 System Design Questions

Q: Design an AI system that can process 1 million PDF documents per day.

A: Async pipeline: Documents uploaded to S3 trigger Pub/Sub events. Cloud Run workers pull from Pub/Sub (fan-out: 100 concurrent workers). Each worker: PDF → images (poppler), images → text (Claude vision or Textract for structured forms), text → chunks (semantic splitter), chunks → embeddings (batched, 512/request via Vertex AI), embeddings → Qdrant. Bottleneck analysis: 1M docs/day = 12 docs/second. Claude vision: ~3s/page, 10 pages avg = 30s/doc. Need 360 concurrent workers. Cost: ~\$0.005/page * 10 pages * 1M = \$50K/day — use Textract for non-complex docs (\$0.0015/page) with Claude fallback for complex.

Q: How would you build a real-time AI coding assistant like GitHub Copilot?

A: Core components: (1) VSCode extension — captures code context (current file, open files, recent edits) and sends to completion API; (2) Context assembler — trims context to 8K tokens: current cursor position + surrounding N lines + relevant snippets from RAG index; (3) RAG index — project-specific code indexed with code-aware embeddings (CodeBERT/UniXcoder); (4) Completion service — vLLM serving a code-specialized model (DeepSeek Coder, CodeLlama) with speculative decoding for low latency; (5) Streaming — server-sent events for token-by-token streaming to IDE. Latency target: first token <200ms, achieved via prefix caching on the prompt prefix.

13.4 LLM Infrastructure & Optimization Questions

Q: Explain vLLM's PagedAttention and why it matters for production inference.

A: Traditional inference allocates a fixed KV cache block per sequence upfront ($\text{max_seq_len} * \text{num_heads} * \text{head_dim} * 2 * \text{dtype_bytes}$). For 100 concurrent sequences with 4K max length on an A100 80GB, this pre-allocates $100 * 4096 * 32 * 128 * 2 * 2$ bytes = 64GB — entire GPU memory. Most sequences finish before max_len , wasting memory. PagedAttention manages KV cache as fixed-size pages (16 tokens/page) in a page table, like OS virtual memory. Pages are allocated on demand and freed on completion. This reduces memory waste from ~70% to <5%, enabling 3-4x higher throughput on the same hardware.

Q: How do you reduce LLM inference costs by 60% without degrading quality?

A: (1) Semantic caching: cache responses for semantically similar queries — Redis with cosine similarity threshold 0.95 — achieves 30-40% cache hit rate; (2) Model routing: classify query complexity, route simple queries to Haiku/Haiku, complex to Opus — 70% simple queries * 10x cost difference = 63% cost reduction; (3) Prompt compression: use LLMingua to compress long contexts by 50% with <5% quality loss; (4) Batching: batch API calls during off-peak; (5) Output caching: cache partial generations for templated responses. Combined, achieves 60-70% cost reduction.

13.5 Leadership & Architecture Questions

Q: Your team wants to use a different LLM framework for every project. How do you handle this?

A: Establish an AI platform layer: (1) Define approved framework list based on production readiness, team expertise, vendor support; (2) Create internal wrappers that abstract framework-specific APIs behind common interfaces — teams build to the interface, not the framework; (3) Implement shared observability, cost tracking, and safety guardrails at the platform layer — applicable regardless of framework; (4) Allow experimentation in sandboxed environments; promote to production only after security review and load testing. Prevent the real risk: 5 teams using 5 frameworks = 5x maintenance burden, 5x security surface.

Q: How do you justify the ROI of a \$500K/year enterprise AI platform to leadership?

A: Quantify on three axes: (1) Cost reduction — if replacing 20 FTE-equivalents of manual work at \$80K average = \$1.6M/year savings; (2) Revenue enablement — faster proposal generation (3 days → 2 hours) enables 2x more RFP responses = pipeline increase; (3) Quality improvement — 40% reduction in hallucinations = fewer client escalations = churn reduction. Present a 3-year NPV: Year 1: -\$500K (build); Year 2: +\$800K net; Year 3: +\$1.2M net. IRR ~80%. Frame against alternative: 3 more engineers at \$200K each = \$600K/year with half the capability.

Must-Know Advanced Topics for Senior AI Engineers

14.1 AI Observability Stack

Production AI systems are inherently non-deterministic. Standard application monitoring (error rates, latency) is necessary but not sufficient. You need AI-specific observability: trace every LLM call, every retrieval, every tool invocation, and evaluate output quality continuously.

Tool	Category	Strengths	Best For
LangSmith	LLM tracing + evaluation	Deep LangChain integration, evaluation chains	LangChain / LangGraph shops
Arize Phoenix	Open-source LLM observability	RAG evaluation, embedding drift	Hybrid / open-source
Weights & Biases	ML experiment tracking	Training metrics, prompt versioning	Fine-tuning teams
Helicone	LLM proxy + analytics	Cost tracking, caching, rate limits	Multi-provider setups
Langfuse	Open-source LLM observability	Self-hostable, prompt management	Privacy-sensitive orgs
OpenTelemetry	Distributed tracing standard	Vendor-neutral, full stack	Enterprise / custom

14.2 vLLM Production Deployment

```
# Deploy vLLM on GKE with GPU # vllm-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-server
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        args:
          - --model=meta-llama/Llama-3-8B-Instruct
          - --tensor-parallel-size=1
          - --max-model-len=8192
          - --max-num-seqs=256
          - --enable-prefix-caching
          # KV cache reuse for repeated prefixes
          - --quantization=awq
          # 4-bit AWQ quantization
          - --served-model-name=llama3-8b
        resources:
          limits:
            nvidia.com/gpu: "1"
          requests:
            memory: "24Gi"
        ports:
          - containerPort: 8000
        nodeSelector:
          cloud.google.com/gke-accelerator: nvidia-l4
      ---
      # Python client with speculative decoding enabled from openai
      import OpenAI
      client = OpenAI(base_url="http://vllm-service:8000/v1", api_key="EMPTY")
      response = client.chat.completions.create(
        model="llama3-8b",
        messages=[{"role": "user", "content": "Explain RAG in 3 sentences."}],
        stream=True,
        temperature=0.1,
        max_tokens=512
      )
      for chunk in response:
        print(chunk.choices[0].delta.content or "", end="", flush=True)
```

14.3 AI Security — Jailbreaks, Prompt Injection & Guardrails

Threat	Attack Vector	Mitigation
Direct Prompt Injection	User crafts input to override system prompt	Input validation, spotlighting (XML tags), repeated instruction

Threat	Attack Vector	Mitigation
Indirect Prompt Injection	Malicious content in retrieved docs/tool results	Sanitise tool outputs, separate trust levels, output monitoring
Jailbreaks	Role-play, encoding tricks to bypass safety	LLM-based classifier on output, Guardrails AI, Llama Guard
Data Exfiltration	Agent leaks private context via tool calls	Output filtering, DLP scanning, tool call logging
Training Data Extraction	Repeated prompts to extract memorised data	Differential privacy in training, output deduplication filter
Model Inversion	Reconstruct training data from model weights	Federated learning, confidential computing

```
# Guardrails AI – input/output validation from guardrails
import Guard from guardrails.hub
import ToxicLanguage, PII, ValidJson
guard = Guard().use_many(
    ToxicLanguage(threshold=0.5, on_fail="exception"),
    PII(pii_entities=["EMAIL_ADDRESS", "PHONE_NUMBER", "SSN"], on_fail="fix"),
)
@guard.validate
def call_llm(prompt: str) -> str:
    return llm.invoke(prompt)

# Llama Guard for safety classification from transformers
import pipeline
safety_classifier = pipeline("text-classification",
    model="meta-llama/LlamaGuard-7b",
    device="cuda")
def is_safe(text: str) -> bool:
    result = safety_classifier(text)[0]
    return result["label"] == "safe"
```

14.4 Prompt Engineering at Scale

- Version prompts like code: Git-tracked, tagged with semantic versions (v1.2.3).
- A/B test prompt changes on production traffic with a feature flag system.
- Use LangSmith / Langfuse for prompt management and evaluation suite tracking.
- Build a prompt eval suite: 50+ test cases with expected outputs; run on every prompt change.
- Use XML tags to structure complex prompts — reduces hallucination by 15-20% vs plain text.
- Implement few-shot examples dynamically: retrieve the N most relevant examples from a vector store.
- For multi-step reasoning: use chain-of-thought with tags and strip before returning to user.

14.5 AI Reliability Engineering (AI-SRE)

SRE Concept	Traditional	AI Systems
SLO	99.9% uptime, <200ms P99	99% task completion, <2s E2E latency, >0.85 faithfulness
Error budget	Downtime minutes/month	Token budget per task, hallucination rate budget
Alerting	CPU/memory/error rate	Faithfulness drop, cost spike, tool failure rate
On-call	Service restart, rollback	Model swap, prompt rollback, RAG index refresh
Chaos engineering	Kill pods, inject latency	Inject bad retrieval, corrupt tool results, token limit
Capacity planning	Request rate * avg latency	Token throughput * model capacity * GPU count

14.6 Kafka for AI Systems

Apache Kafka enables decoupled, scalable, event-driven AI architectures. Key patterns for AI engineers:

- Document ingestion pipeline: document-uploads topic → consumer group of embedding workers → embeddings stored in vector DB.
- Agent event bus: agents publish actions to an agent-events topic; human review service subscribes to high-risk action events.
- Async tool execution: LLM tool calls produce to tool-requests topic; tool servers consume and publish results to tool-results.
- Model monitoring: inference requests/responses streamed to a monitoring topic; Flink job computes drift metrics in real-time.
- Multi-tenant ingestion: partition by tenant_id for isolation; set per-tenant consumer group quotas.

14.7 CI/CD for LLMs — LLMOps

```
# .github/workflows/llm-ci.yaml name: LLM CI/CD Pipeline on: [push] jobs: eval: runs-on:
ubuntu-latest steps: - uses: actions/checkout@v4 - name: Run prompt evaluation suite run:
| python eval/run_evals.py \ --prompt-dir prompts/ \ --test-suite eval/test_cases.json \
--model claude-sonnet-4-6 \ --threshold 0.85 env: ANTHROPIC_API_KEY: ${
secrets.ANTHROPIC_API_KEY }} - name: RAG retrieval quality check run: python
eval/rag_eval.py --min-ndcg 0.75 - name: Safety scan (Llama Guard) run: python
eval/safety_scan.py --fail-on-unsafe - name: Cost projection check run: python
eval/cost_check.py --max-cost-per-query 0.05 - name: Deploy to staging if: success()
run: kubectl apply -f k8s/staging/ - name: Integration tests on staging run: pytest
tests/integration/ -v - name: Promote to production if: success() && github.ref ==
'refs/heads/main' run: | kubectl apply -f k8s/production/ kubectl rollout status
deployment/ai-service
```

APPENDIX — Quick Reference Cards

A.1 RAG Quick Checklist

- ■ Chunking strategy tested against retrieval quality metrics (NDCG, MRR)
- ■ Hybrid search (dense + BM25) implemented and fused via RRF
- ■ Cross-encoder reranker deployed (Cohere rerank or BGE)
- ■ Query rewriting / HyDE expansion for ambiguous queries
- ■ Semantic caching (Redis) to reduce costs
- ■ Faithfulness and relevance metrics tracked in production
- ■ Multi-tenant isolation: separate namespaces or collections
- ■ Incremental indexing pipeline for document updates
- ■ Context window compression for long retrieved documents

A.2 Agent Safety Checklist

- ■ Max iteration limits on all agent loops
- ■ Tool call rate limiting per thread
- ■ Input/output sanitisation against prompt injection
- ■ Human-in-the-loop for any destructive or irreversible action
- ■ Immutable audit log of all agent decisions and tool calls
- ■ Kill switch API to terminate any agent thread
- ■ Cost budget per task with hard cutoff
- ■ Sandbox execution for code tools (Docker/gVisor)

A.3 Interview Day Tips

- Always clarify requirements before designing: scale, latency, consistency, budget.
- Start with a simple design, then iteratively add complexity when asked.
- Quantify everything: 'this handles ~500 RPS given 50ms avg latency per request'.
- Discuss tradeoffs explicitly — interviewers want to see architectural thinking, not memorised solutions.
- When asked about a technology you don't know: say so honestly, then reason from first principles.
- For coding questions: think aloud, discuss edge cases before coding, test with examples.
- System design: draw the architecture first, then walk through data flow, then discuss failure modes.

